End-to-End Data Management Pipeline for a Recommendation System

Data Management for Machine Learning – Assignment 2

GitHub repository: https://github.com/2025ag05005-max/recommendation-system-pipeline

Student Name: Abhinandan M S

BITS ID: 2025AG05005

Email: 2025ag05005@wilp.bits-pilani.ac.in

1. Objective and dataset

The project implements a reproducible recommendation data pipeline using the public MovieLens Latest Small dataset. It ingests user, movie and rating-interaction data; validates data quality; prepares chronological train/test data; creates reusable features; trains two recommenders; evaluates them; and orchestrates the workflow.

MovieLens provides movie metadata and timestamped ratings. Since the dataset does not provide demographic user profiles, the users entity is derived from distinct observed user identifiers and records its source. No demographic attributes are fabricated.

2. Architecture and implementation

The pipeline stages are: ingestion → validation → preparation → transformation → feature management → model training → model evaluation → orchestration. Python and Pandas handle ingestion and data processing; validation checks cover nulls, duplicates, types, rating range, timestamp validity and referential integrity. Invalid interactions are separated into an error CSV and validation results are saved as JSON and CSV.

A chronological per-user holdout is used: for users with at least two interactions, the latest interaction is assigned to test and earlier interactions are retained for training. User activity, item popularity and a user-item matrix are generated. Reusable user, item and interaction feature tables are documented in a feature catalog.

3. Models and evaluation

Two models are trained: (1) a popularity-based baseline ranked by training interaction count and average rating, and (2) personalized item-based collaborative filtering using cosine similarity between item interaction vectors. Ranking relevance is defined as a held-out rating of at least 4.0. Evaluation uses K=5 and reports Precision@5, Recall@5, NDCG@5, catalog coverage, and intra-list genre diversity. Explicit rating predictions are additionally evaluated using RMSE and MAE.

4. Experiment tracking and orchestration

MLflow records model names, K, relevance threshold, training/test sizes, evaluation metrics and the validation report artifact. A dependency-aware Python workflow executes stages in sequence, stops when a dependency fails, and writes a JSON execution summary. The README documents environment setup, execution, output locations and the MLflow UI.

5. Results

Model	precision_at_5	recall_at_5	ndcg_at_5	coverage	diversity	rmse	mae	eligible_users	evaluated_test_ratings
Popularity	0.0092	0.0364	0.0256	0.0062	0.7733	1.0743	0.8568	391.0	889.0
ItemBasedCollaborativeFiltering	0.0	0.0	0.0	0.047	0.7686	1.0332	0.8046	391.0	889.0

Interpretation: compare the baseline and personalized model across ranking quality, catalog reach, recommendation variety and rating prediction error. Do not claim one model is superior unless the recorded results support that conclusion.

6. Limitations

MovieLens Latest Small is a compact benchmark and does not represent all production environments. The latest-one-event holdout is a simple temporal evaluation design. Genre-based diversity is a proxy for variety. The dense item-similarity matrix may require a sparse or approximate-neighbor implementation for substantially larger catalogs.

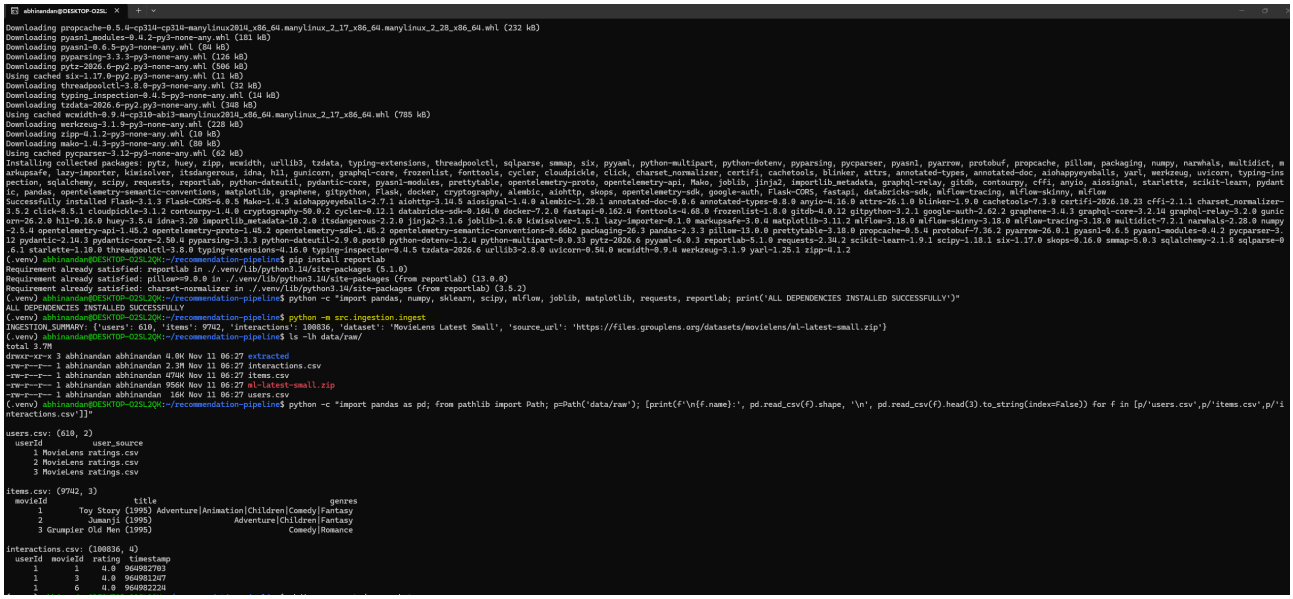
7. Reproducibility

From the repository root, install dependencies with pip install -r requirements.txt and execute python run_pipeline.py. The workflow downloads the public dataset on first run, writes intermediate and final outputs to the documented folders, tracks experiments in MLflow, and records orchestration status in logs/pipeline_run.json.

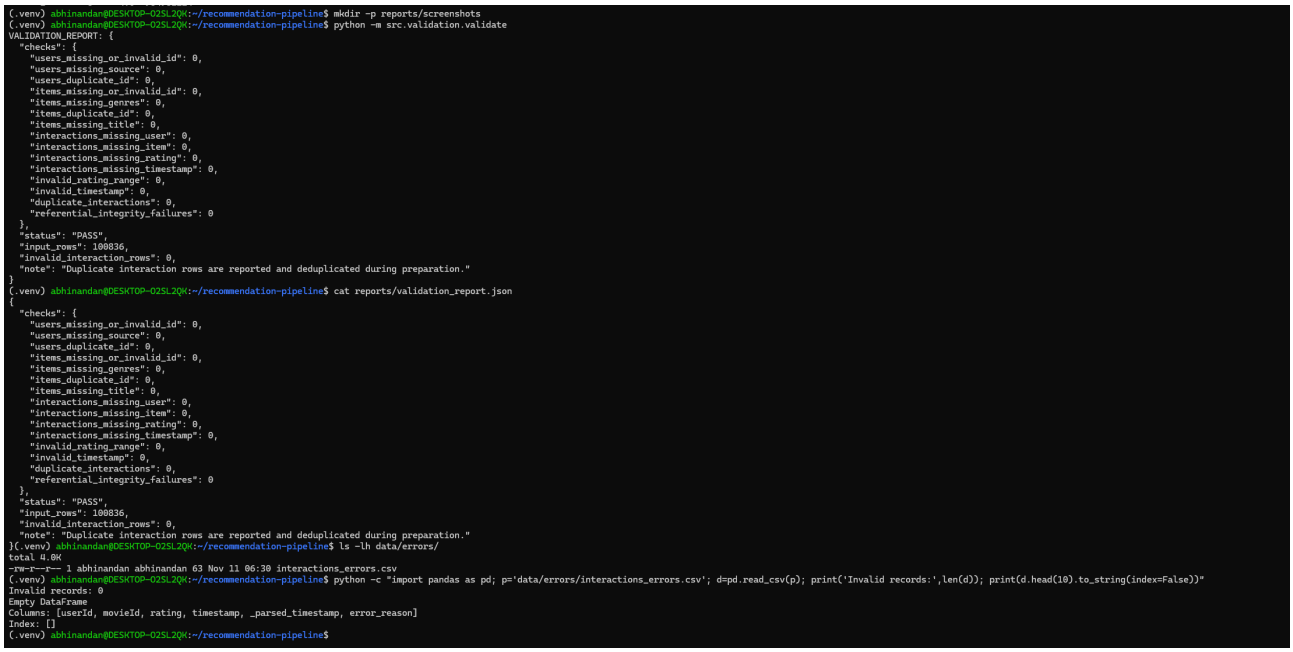
Execution Evidence – Screenshots

The following appendix contains screenshots captured by the student from the actual execution environment. Each image should show the corresponding successful stage or output.

1. Data ingestion



2. Data validation



3. Data preparation and temporal split



4. Data transformation

```
(.venv) abhinandan@DESKTOP-O25L2QK:~/recommendation-pipeline$ python - <<'PY'
import pandas as pd

print("=" * 65)
print("STAGE 4: DATA TRANSFORMATION — EXECUTION EVIDENCE")
print("=" * 65)

# 1. User activity
df = pd.read_csv("data/features/user_activity_summary.csv")
print("\n1. USER ACTIVITY SUMMARY")
print("Shape:", df.shape)
print(df.head(5).to_string(index=False))

# 2. Item popularity
df = pd.read_csv("data/features/item_popularity.csv")
print("\n2. ITEM POPULARITY")
print("Shape:", df.shape)
print(df.head(5).to_string(index=False))

# 3. User-item matrix
df = pd.read_csv("data/features/user_item_matrix.csv", index_col=0)
print("\n3. USER-ITEM INTERACTION MATRIX")
print("Shape:", df.shape)
print("Number of users:", df.shape[0])
print("Number of movie columns:", df.shape[1])
print("Non-zero ratings:", int((df != 0).sum().sum()))
print("Sparsity:", round((df == 0).sum().sum() / df.size * 100, 2), "%")

print("\nFirst 5 users and first 8 movie columns:")
print(df.iloc[:5, :8].to_string())

print("\n" + "=" * 65)
print("TRANSFORMATION OUTPUT VERIFICATION COMPLETED")
print("=" * 65)
PY
=================================================================
STAGE 4: DATA TRANSFORMATION — EXECUTION EVIDENCE
=================================================================

1. USER ACTIVITY SUMMARY
Shape: (610, 4)
 userId  activity_count  user_avg_rating    last_interaction
      1             231         4.367965 2000-07-30 19:09:36
      2              28         3.928571 2015-10-24 19:34:36
      3              38         2.486842 2011-05-27 02:44:44
      4             215         3.553488 2001-12-05 17:48:52
      5              41         3.609756 1996-11-08 06:41:51

2. ITEM POPULARITY
Shape: (9742, 5)
 movieId                               title                                        genres  interaction_count  item_avg_rating
       1                    Toy Story (1995) Adventure|Animation|Children|Comedy|Fantasy              214.0         3.915888
       2                      Jumanji (1995)                   Adventure|Children|Fantasy              107.0         3.425234
       3             Grumpier Old Men (1995)                               Comedy|Romance               50.0         3.250000
       4            Waiting to Exhale (1995)                         Comedy|Drama|Romance                7.0         2.357143
       5  Father of the Bride Part II (1995)                                       Comedy               48.0         3.072917

3. USER-ITEM INTERACTION MATRIX
Shape: (610, 9695)
Number of users: 610
Number of movie columns: 9695
Non-zero ratings: 99947
Sparsity: 98.31 %

First 5 users and first 8 movie columns:
          1    2    3    4    5    6    7    8
userId
1       4.0  0.0  4.0  0.0  0.0  4.0  0.0  0.0
2       0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0
3       0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0
4       0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0
5       4.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0

=================================================================
TRANSFORMATION OUTPUT VERIFICATION COMPLETED
=================================================================
(.venv) abhinandan@DESKTOP-O25L2QK:~/recommendation-pipeline$
```

5. Feature management

```
(.venv) abhinandan@DESKTOP-O25L2QK:~/recommendation-pipeline$ python -m src.features.build_features
FEATURE_BUILD_SUMMARY: {'user_features': 610, 'item_features': 9742, 'interaction_features': 99947, 'catalog_items': 9742, 'items_without_training_interactions': 47, 'feature_catalog_entries': 10}
(.venv) abhinandan@DESKTOP-O25L2QK:~/recommendation-pipeline$ python - <<'PY'
import pandas as pd

items = pd.read_csv("data/processed/items_clean.csv")
features = pd.read_csv("data/features/item_features.csv")
catalog = pd.read_csv("data/features/feature_catalog.csv")

print("=" * 55)
print("CORRECTED ITEM FEATURE VALIDATION")
print("=" * 55)

print("Catalog items:", len(items))
print("Feature records:", len(features))
print("Missing catalog items:",
      len(set(items.movieId) - set(features.movieId)))
print("Duplicate item IDs:", features.movieId.duplicated().sum())
print("Items with interactions:",
      int(features.has_training_interactions.sum()))
print("Items without interactions:",
      int((~features.has_training_interactions).sum()))
print("Zero popularity items:",
      int((features.item_popularity == 0).sum()))
print("Missing average ratings:",
      int(features.item_average_rating.isna().sum()))
print("Feature catalog entries:", len(catalog))

print("\nSample unrated movies:")
print(
    features.loc[
        features.item_popularity == 0,
        ["movieId", "title", "genres",
         "item_popularity", "item_average_rating",
         "has_training_interactions"]
    ].head(5).to_string(index=False)
)

assert len(features) == len(items)
assert features.movieId.nunique() == len(items)
assert features.movieId.duplicated().sum() == 0
assert (features.item_popularity == 0).sum() == 47
assert (~features.has_training_interactions).sum() == 47

print("\nCATALOG COVERAGE VALIDATION: PASS")
PY
=======================================================
CORRECTED ITEM FEATURE VALIDATION
=======================================================
Catalog items: 9742
Feature records: 9742
Missing catalog items: 0
Duplicate item IDs: 0
Items with interactions: 9695
Items without interactions: 47
Zero popularity items: 47
Missing average ratings: 47
Feature catalog entries: 10

Sample unrated movies:
 movieId                                title                  genres  item_popularity  item_average_rating  has_training_interactions
     178             Love & Human Remains (1993)          Comedy|Drama                0                  NaN                      False
     486                   Federal Hill (1994)                   Drama                0                  NaN                      False
     496            What Happened Was... (1994) Comedy|Drama|Romance|Thriller           0                  NaN                      False
     626 Thin Line Between Love and Hate, A (1996)                  Comedy                0                  NaN                      False
     706                       Sunset Park (1996)                   Drama                0                  NaN                      False

CATALOG COVERAGE VALIDATION: PASS
(.venv) abhinandan@DESKTOP-O25L2QK:~/recommendation-pipeline$
```

6. Model training

```
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$ python -c "from src.training.models import train_models; train_models()"
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$ ls -lh --time-style=long-iso models/
total 349M
-rw-r--r-- 1 abhinandan abhinandan 348M 2026-11-11 07:15 item_cf_model.joblib
-rw-r--r-- 1 abhinandan abhinandan 653K 2026-11-11 07:15 popularity_model.joblib
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$ python - <<'PY'
from src.training.models import train_models

popularity, item_cf, train, items = train_models()

print("\nMODEL TRAINING COMPLETED")
print("Popularity model:", type(popularity).__name__)
print("Collaborative filtering model:", type(item_cf).__name__)
print("Training interactions:", len(train))
print("Catalog items:", len(items))
print("MODEL TRAINING: SUCCESS")
PY

MODEL TRAINING COMPLETED
Popularity model: PopularityRecommender
Collaborative filtering model: ItemCFRecommender
Training interactions: 99947
Catalog items: 9742
MODEL TRAINING: SUCCESS
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$ python - <<'PY'
import joblib

for name in ["popularity_model.joblib", "item_cf_model.joblib"]:
    model = joblib.load("models/" + name)
    print(name, "LOADED SUCCESSFULLY")
    print("Class:", type(model).__name__)
    print("Module:", type(model).__module__)

print("\nMODEL ARTIFACT VERIFICATION: PASS")
PY
popularity_model.joblib LOADED SUCCESSFULLY
Class: PopularityRecommender
Module: src.training.models
item_cf_model.joblib LOADED SUCCESSFULLY
Class: ItemCFRecommender
Module: src.training.models

MODEL ARTIFACT VERIFICATION: PASS
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$
```

7. Recommendation generation

```
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$ python - <<'PY'
import joblib
import pandas as pd

train = pd.read_csv("data/processed/train_interactions.csv")

popularity = joblib.load("models/popularity_model.joblib")
item_cf = joblib.load("models/item_cf_model.joblib")

user_id = int(train.userId.iloc[0])

seen = set(
    train.loc[train.userId == user_id, "movieId"]
)

print("=" * 60)
print("STAGE 6: RECOMMENDATION GENERATION VERIFICATION")
print("=" * 60)

print("Test User:", user_id)
print("Previously Watched Movies:", len(seen))

for name, model in [
    ("Popularity", popularity),
    ("Item Collaborative Filtering", item_cf)
]:
    recommendations = model.recommend(user_id, seen, k=5)

    print("\nMODEL:", name)
    print("Recommended Movie IDs:", recommendations)
    print("Recommendation Count:", len(recommendations))
    print(
        "Previously Watched Movies Recommended:",
        len(set(recommendations) & seen)
    )

    assert len(recommendations) == 5
    assert len(set(recommendations) & seen) == 0

print("\nRECOMMENDATION GENERATION: PASS")
PY
============================================================
STAGE 6: RECOMMENDATION GENERATION VERIFICATION
============================================================
Test User: 1
Previously Watched Movies: 231

MODEL: Popularity
Recommended Movie IDs: [318, 589, 4993, 858, 150]
Recommendation Count: 5
Previously Watched Movies Recommended: 0

MODEL: Item Collaborative Filtering
Recommended Movie IDs: [131724, 1140, 3106, 6158, 26366]
Recommendation Count: 5
Previously Watched Movies Recommended: 0

RECOMMENDATION GENERATION: PASS
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$
```

8. Model evaluation

```
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$ grep -n "set_tracking_uri" src/evaluation/evaluate.py
84:    mlflow.set_tracking_uri("sqlite:///mlflow.db")
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$ python -m src.evaluation.evaluate
2026/11/11 07:25:55 INFO mlflow.store.db.utils: Creating initial MLflow database tables...
2026/11/11 07:25:55 INFO mlflow.store.db.utils: Updating database tables
MODEL_EVALUATION: {
  "Popularity": {
    "precision_at_5": 0.009207161125319694,
    "recall_at_5": 0.03640349420400571,
    "ndcg_at_5": 0.025593778300112557,
    "coverage": 0.006158899609936358,
    "diversity": 0.7732768865121806,
    "rmse": 1.0743287145482574,
    "mae": 0.8567555004952836,
    "eligible_users": 391,
    "evaluated_test_ratings": 889
  },
  "ItemBasedCollaborativeFiltering": {
    "precision_at_5": 0.0,
    "recall_at_5": 0.0,
    "ndcg_at_5": 0.0,
    "coverage": 0.047012933689180865,
    "diversity": 0.7685957861405432,
    "rmse": 1.033182613432211,
    "mae": 0.804645625029475,
    "eligible_users": 391,
    "evaluated_test_ratings": 889
  }
}
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$
```

9. MLflow experiment verification

```
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$ python - <<'PY'
import mlflow

mlflow.set_tracking_uri("sqlite:///mlflow.db")

experiments = mlflow.search_experiments()

print("MLFLOW EXPERIMENT VERIFICATION")
for exp in experiments:
    print("Experiment:", exp.name)
    print("Experiment ID:", exp.experiment_id)

print("\nMLFLOW VERIFICATION: PASS" if experiments else "NO EXPERIMENTS FOUND")
PY
MLFLOW EXPERIMENT VERIFICATION
Experiment: Default
Experiment ID: 0

MLFLOW VERIFICATION: PASS
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$ python - <<'PY'
import mlflow

mlflow.set_tracking_uri("sqlite:///mlflow.db")

runs = mlflow.search_runs(
    experiment_ids=["0"],
    output_format="pandas"
)

print("=" * 60)
print("MLFLOW RUN VERIFICATION")
print("=" * 60)

print("Total runs:", len(runs))

for _, run in runs.iterrows():
    print("\nRun name:", run.get("tags.mlflow.runName"))
    print("Run ID:", run["run_id"])

    for metric in [
        "precision_at_5",
        "recall_at_5",
        "ndcg_at_5",
        "rmse",
        "mae"
    ]:
        if metric in runs.columns:
            print(metric, ":", run.get("metrics." + metric))

print("\nMLFLOW RUN VERIFICATION COMPLETED")
PY
============================================================
MLFLOW RUN VERIFICATION
============================================================
Total runs: 2

Run name: ItemBasedCollaborativeFiltering
Run ID: d2ddf80dd8c6437cb000c66a47b35692

Run name: Popularity
Run ID: 80137462de6743d3a5f4396dbe744697

MLFLOW RUN VERIFICATION COMPLETED
```

10. Ranking diagnostics

```
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$ python - <<'PY'
import joblib
import pandas as pd

train = pd.read_csv("data/processed/train_interactions.csv")
test = pd.read_csv("data/processed/test_interactions.csv")

models = {
    "Popularity": joblib.load("models/popularity_model.joblib"),
    "ItemCF": joblib.load("models/item_cf_model.joblib")
}

train_seen = train.groupby("userId")["movieId"].apply(set).to_dict()

relevant = (
    test[test["rating"] >= 4.0]
    .groupby("userId")["movieId"]
    .apply(set)
    .to_dict()
)

print("=" * 60)
print("RANKING METRIC DIAGNOSTICS")
print("=" * 60)

print("Test users:", test.userId.nunique())
print("Users with relevant test movies:", len(relevant))

for name, model in models.items():
    print("\nMODEL:", name)
    total_hits = 0

    for user_id in list(relevant.keys())[:10]:
        seen = train_seen.get(user_id, set())
        recs = model.recommend(int(user_id), seen, k=5)
        hits = set(recs) & relevant[user_id]
        total_hits += len(hits)

        print(
            "User:", user_id,
            "| Relevant:", len(relevant[user_id]),
            "| Recommendations:", recs,
            "| Hits:", len(hits)
        )

    print("Hits in first 10 users:", total_hits)

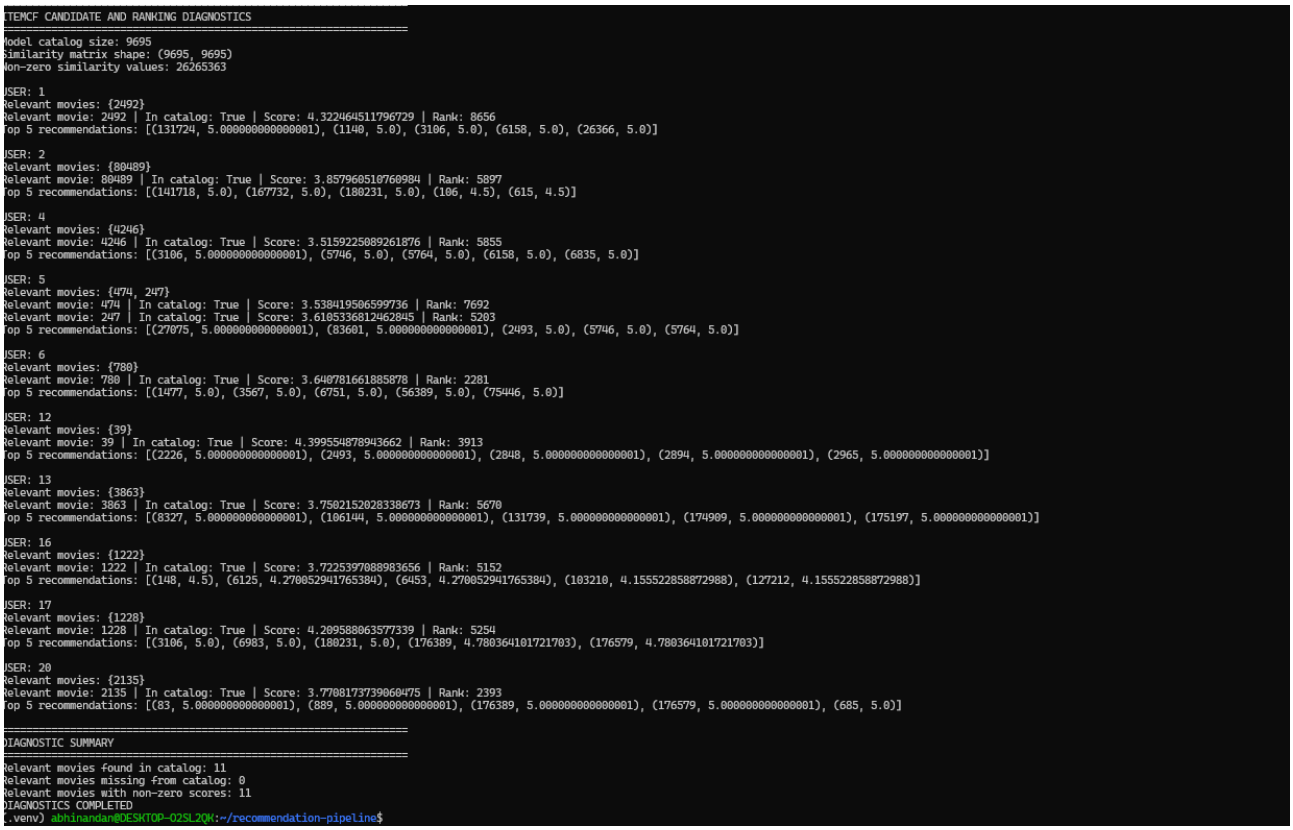
print("\nDIAGNOSTIC COMPLETED")
PY
============================================================
RANKING METRIC DIAGNOSTICS
============================================================
Test users: 610
Users with relevant test movies: 391

MODEL: Popularity
User: 1 | Relevant: 1 | Recommendations: [318, 589, 4993, 858, 150] | Hits: 0
User: 2 | Relevant: 1 | Recommendations: [356, 296, 2571, 593, 260] | Hits: 0
User: 4 | Relevant: 1 | Recommendations: [318, 356, 110, 527, 480] | Hits: 0
User: 5 | Relevant: 2 | Recommendations: [356, 2571, 593, 260, 2959] | Hits: 0
User: 6 | Relevant: 1 | Recommendations: [2571, 260, 2959, 1196, 1] | Hits: 0
User: 12 | Relevant: 1 | Recommendations: [318, 356, 296, 2571, 593] | Hits: 0
User: 13 | Relevant: 1 | Recommendations: [318, 356, 296, 593, 260] | Hits: 0
User: 16 | Relevant: 1 | Recommendations: [110, 480, 589, 1, 1210] | Hits: 0
User: 17 | Relevant: 1 | Recommendations: [457, 608, 32, 588, 364] | Hits: 0
User: 20 | Relevant: 1 | Recommendations: [318, 356, 296, 2571, 593] | Hits: 0
Hits in first 10 users: 0

MODEL: ItemCF
User: 1 | Relevant: 1 | Recommendations: [131724, 1140, 3106, 6158, 26366] | Hits: 0
User: 2 | Relevant: 1 | Recommendations: [141718, 167732, 180231, 106, 615] | Hits: 0
User: 4 | Relevant: 1 | Recommendations: [3106, 5746, 5764, 6158, 6835] | Hits: 0
User: 5 | Relevant: 2 | Recommendations: [27075, 83601, 2493, 5746, 5764] | Hits: 0
User: 6 | Relevant: 1 | Recommendations: [1477, 3567, 6751, 56389, 75446] | Hits: 0
User: 12 | Relevant: 1 | Recommendations: [2226, 2493, 2848, 2894, 2965] | Hits: 0
User: 13 | Relevant: 1 | Recommendations: [8327, 106144, 131739, 174909, 175197] | Hits: 0
User: 16 | Relevant: 1 | Recommendations: [148, 6125, 6453, 103210, 127212] | Hits: 0
User: 17 | Relevant: 1 | Recommendations: [3106, 6983, 180231, 176389, 176579] | Hits: 0
User: 20 | Relevant: 1 | Recommendations: [83, 889, 176389, 176579, 685] | Hits: 0
Hits in first 10 users: 0

DIAGNOSTIC COMPLETED
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$
```

11. Item collaborative filtering diagnostics



12. Item collaborative filtering ranking analysis

```
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$
python - <<'PY'
import joblib
import pandas as pd
import numpy as np

train = pd.read_csv("data/processed/train_interactions.csv")
test = pd.read_csv("data/processed/test_interactions.csv")

model = joblib.load("models/item_cf_model.joblib")

train_seen = train.groupby("userId")["movieId"].apply(set).to_dict()

relevant = (
    test[test["rating"] >= 4.0]
    .groupby("userId")["movieId"]
    .apply(set)
    .to_dict()
)

ranks = []

for user_id, relevant_movies in relevant.items():

    seen = train_seen.get(user_id, set())
    scores = model._scores(int(user_id))

    ranked = sorted(
        [(mid, score) for mid, score in scores.items()
         if mid not in seen],
        key=lambda x: (-x[1], x[0])
    )

    rank_map = {
        movie_id: position
        for position, (movie_id, _) in enumerate(ranked, 1)
    }

    for movie_id in relevant_movies:
        if movie_id in rank_map:
            ranks.append(rank_map[movie_id])

ranks = np.array(ranks)

print("=" * 60)
print("ITEMCF FULL RANKING ANALYSIS")
print("=" * 60)

print("Total relevant movies:", len(ranks))
print("Best rank:", ranks.min())
print("Worst rank:", ranks.max())
print("Average rank:", round(ranks.mean(), 2))
print("Median rank:", np.median(ranks))

for k in [5, 10, 20, 50, 100, 500, 1000]:
    print(f"Relevant movies within Top {k}:",
          int((ranks <= k).sum()))

print("\nRANKING ANALYSIS COMPLETED")
PY
============================================================
ITEMCF FULL RANKING ANALYSIS
============================================================
Total relevant movies: 495
Best rank: 8
Worst rank: 9569
Average rank: 4199.41
Median rank: 3913.0
Relevant movies within Top 5: 0
Relevant movies within Top 10: 1
Relevant movies within Top 20: 1
Relevant movies within Top 50: 1
Relevant movies within Top 100: 2
Relevant movies within Top 500: 5
Relevant movies within Top 1000: 17

RANKING ANALYSIS COMPLETED
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$
```

13. Item collaborative filtering rating diagnostics

```
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$
python - <<'PY'
import joblib
import pandas as pd

train = pd.read_csv("data/processed/train_interactions.csv")
test = pd.read_csv("data/processed/test_interactions.csv")

model = joblib.load("models/item_cf_model.joblib")

print("=" * 65)
print("ITEMCF RATING PREDICTION DIAGNOSTICS")
print("=" * 65)

sample = test.head(20)

for row in sample.itertuples(index=False):
    predicted = model.predict_rating(
        int(row.userId),
        int(row.movieId)
    )

    print(
        f"User: {row.userId:<5} "
        f"Movie: {row.movieId:<7} "
        f"Actual: {row.rating:<5} "
        f"Predicted: {predicted:.4f}"
    )

print("\nRATING DIAGNOSTICS COMPLETED")
PY
=================================================================
ITEMCF RATING PREDICTION DIAGNOSTICS
=================================================================
User: 1     Movie: 2492    Actual: 4.0   Predicted: 4.3225
User: 2     Movie: 80489   Actual: 4.5   Predicted: 3.8580
User: 3     Movie: 2424    Actual: 0.5   Predicted: 1.4275
User: 4     Movie: 4246    Actual: 4.0   Predicted: 3.5159
User: 5     Movie: 247     Actual: 5.0   Predicted: 3.6105
User: 5     Movie: 300     Actual: 3.0   Predicted: 3.5412
User: 5     Movie: 474     Actual: 4.0   Predicted: 3.5384
User: 6     Movie: 780     Actual: 5.0   Predicted: 3.6408
User: 7     Movie: 49286   Actual: 0.5   Predicted: 3.2116
User: 8     Movie: 141     Actual: 3.0   Predicted: 3.5977
User: 8     Movie: 235     Actual: 3.0   Predicted: 3.6343
User: 9     Movie: 5962    Actual: 1.0   Predicted: 3.5005
User: 10    Movie: 1088    Actual: 3.0   Predicted: 3.2612
User: 11    Movie: 368     Actual: 3.0   Predicted: 3.7996
User: 12    Movie: 39      Actual: 4.0   Predicted: 4.3996
User: 13    Movie: 3863    Actual: 4.0   Predicted: 3.7502
User: 14    Movie: 784     Actual: 1.0   Predicted: 3.3566
User: 15    Movie: 63859   Actual: 3.0   Predicted: 3.3982
User: 16    Movie: 1222    Actual: 4.5   Predicted: 3.7225
User: 17    Movie: 1228    Actual: 4.5   Predicted: 4.2096

RATING DIAGNOSTICS COMPLETED
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$
```

14. Successful orchestration run

```
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$ sed -i 's|mlflow.set_tracking_uri("file:./mlruns")|mlflow.set_tracking_uri("sqlite:///mlflow.db")|' src/orchestration/pipeline.py
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$ grep -n "set_tracking_uri" src/orchestration/pipeline.py
16:    mlflow.set_tracking_uri("sqlite:///mlflow.db")
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$ python -m src.orchestration.pipeline

========= START: ingestion =========
INGESTION_SUMMARY: {'users': 610, 'items': 9742, 'interactions': 100836, 'dataset': 'MovieLens Latest Small', 'source_url': 'https://files.grouplens.org/datasets/movielens/ml-latest-small.zip'}
========= SUCCESS: ingestion (0.1s) =========

========= START: validation =========
VALIDATION_REPORT: {
  "checks": {
    "users_missing_or_invalid_id": 0,
    "users_missing_source": 0,
    "users_duplicate_id": 0,
    "items_missing_or_invalid_id": 0,
    "items_missing_genres": 0,
    "items_duplicate_id": 0,
    "items_missing_title": 0,
    "interactions_missing_user": 0,
    "interactions_missing_item": 0,
    "interactions_missing_rating": 0,
    "interactions_missing_timestamp": 0,
    "invalid_rating_range": 0,
    "invalid_timestamp": 0,
    "duplicate_interactions": 0,
    "referential_integrity_failures": 0
  },
  "status": "PASS",
  "input_rows": 100836,
  "invalid_interaction_rows": 0,
  "note": "Duplicate interaction rows are reported and deduplicated during preparation."
}
========= SUCCESS: validation (0.0s) =========

========= START: preparation =========
{'clean_interactions': 100836, 'train_rows': 99947, 'test_rows': 889, 'users': 610, 'items': 9742, 'train_max_timestamp': '2018-09-24 02:44:19', 'test_min_timestamp': '1996-03-29 18:39:22'}
========= SUCCESS: preparation (0.3s) =========

========= START: transformation =========
{'user_activity_rows': 610, 'item_popularity_rows': 9742, 'user_item_matrix_shape': (610, 9695)}
========= SUCCESS: transformation (1.1s) =========

========= START: feature_management =========
FEATURE_BUILD_SUMMARY: {'user_features': 610, 'item_features': 9742, 'interaction_features': 99947, 'catalog_items': 9742, 'items_without_training_interactions': 47, 'feature_catalog_entries': 10}
========= SUCCESS: feature_management (0.3s) =========

========= START: model_training =========
========= SUCCESS: model_training (0.0s) =========

========= START: model_evaluation =========
MODEL_EVALUATION: {
  "Popularity": {
    "precision_at_5": 0.009207161125319694,
    "recall_at_5": 0.03640349420400571,
    "ndcg_at_5": 0.025593778300112557,
    "coverage": 0.006158899609936358,
    "diversity": 0.7732768865121806,
    "rmse": 1.0743287145482574,
    "mae": 0.8567555004952836,
    "eligible_users": 391,
    "evaluated_test_ratings": 889
  },
  "ItemBasedCollaborativeFiltering": {
    "precision_at_5": 0.0,
    "recall_at_5": 0.0,
    "ndcg_at_5": 0.0,
    "coverage": 0.047012933689180865,
    "diversity": 0.7685957861405432,
    "rmse": 1.033182613432211,
    "mae": 0.804645625029475,
    "eligible_users": 391,
    "evaluated_test_ratings": 889
  }
}
========= SUCCESS: model_evaluation (24.2s) =========

PIPELINE_RUN_SUMMARY: {
  "status": "SUCCESS",
  "started_utc": "2026-11-11T08:06:54.378599+00:00",
  "finished_utc": "2026-11-11T08:07:21.140835+00:00",
  "tasks": {
    "ingestion": "SUCCESS",
    "validation": "SUCCESS",
    "preparation": "SUCCESS",
    "transformation": "SUCCESS",
    "feature_management": "SUCCESS",
    "model_training": "SUCCESS",
    "model_evaluation": "SUCCESS"
  }
}
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$
```

15. Evaluation report verification

```
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$
python - <<'PY'
import pandas as pd
import json

with open("reports/evaluation_results.json") as f:
    results = json.load(f)

print("=" * 60)
print("FINAL MODEL EVALUATION SUMMARY")
print("=" * 60)

for model, metrics in results.items():
    print("\nMODEL:", model)
    for key, value in metrics.items():
        print(f"{key}: {value}")

print("\nEVALUATION REPORT VERIFICATION")

df = pd.read_csv("reports/model_comparison.csv")

print("Comparison rows:", len(df))
print("Comparison columns:", list(df.columns))
print(df.to_string(index=False))

print("\nREPORT VERIFICATION: PASS")
PY
============================================================
FINAL MODEL EVALUATION SUMMARY
============================================================

MODEL: Popularity
precision_at_5: 0.009207161125319694
recall_at_5: 0.03640349420400571
ndcg_at_5: 0.025593778300112557
coverage: 0.006158899609936358
diversity: 0.7732768865121806
rmse: 1.0743287145482574
mae: 0.8567555004952836
eligible_users: 391
evaluated_test_ratings: 889

MODEL: ItemBasedCollaborativeFiltering
precision_at_5: 0.0
recall_at_5: 0.0
ndcg_at_5: 0.0
coverage: 0.047012933689180865
diversity: 0.7685957861405432
rmse: 1.033182613432211
mae: 0.804645625029475
eligible_users: 391
evaluated_test_ratings: 889

EVALUATION REPORT VERIFICATION
Comparison rows: 2
Comparison columns: ['Unnamed: 0', 'precision_at_5', 'recall_at_5', 'ndcg_at_5', 'coverage', 'diversity', 'rmse', 'mae', 'eligible_users', 'evaluated_test_ratings']
                     Unnamed: 0  precision_at_5  recall_at_5  ndcg_at_5  coverage  diversity     rmse      mae  eligible_users  evaluated_test_ratings
                     Popularity        0.009207     0.036403   0.025594  0.006159   0.773277 1.074329 0.856756           391.0                   889.0
ItemBasedCollaborativeFiltering        0.000000     0.000000   0.000000  0.047013   0.768596 1.033183 0.804646           391.0                   889.0

REPORT VERIFICATION: PASS
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$
```

16. Complete pipeline execution log

```
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$ cat logs/pipeline_run.json
{
  "status": "SUCCESS",
  "started_utc": "2026-11-11T08:06:54.378599+00:00",
  "finished_utc": "2026-11-11T08:07:21.140835+00:00",
  "tasks": {
    "ingestion": "SUCCESS",
    "validation": "SUCCESS",
    "preparation": "SUCCESS",
    "transformation": "SUCCESS",
    "feature_management": "SUCCESS",
    "model_training": "SUCCESS",
    "model_evaluation": "SUCCESS"
  }
}(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$ ls -lh logs/
total 4.0K
-rw-r--r-- 1 abhinandan abhinandan 374 Nov 11 08:07 pipeline_run.json
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$
```

17. MLflow experiment runs and metrics

```
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$ python - <<'PY'
import mlflow

mlflow.set_tracking_uri("sqlite:///mlflow.db")
runs = mlflow.search_runs(experiment_ids=["0"])

print("MLFLOW RUN VERIFICATION")
print("Total runs:", len(runs))

for _, run in runs.iterrows():
    print("\nModel:", run.get("tags.mlflow.runName"))
    print("Run ID:", run["run_id"])
    for metric in ["precision_at_5", "recall_at_5", "ndcg_at_5", "rmse", "mae"]:
        if "metrics." + metric in runs.columns:
            print(metric, ":", run["metrics." + metric])
PY
MLFLOW RUN VERIFICATION
Total runs: 6

Model: ItemBasedCollaborativeFiltering
Run ID: 3232796495ab41d492bfe92942309dfb
precision_at_5 : 0.0
recall_at_5 : 0.0
ndcg_at_5 : 0.0
rmse : 1.033182613432211
mae : 0.804645625029475

Model: Popularity
Run ID: 1338a913898e4c38b49a1f0391f3b93a
precision_at_5 : 0.009207161125319694
recall_at_5 : 0.036403494204005 71
ndcg_at_5 : 0.025593778300112557
rmse : 1.0743287145482574
mae : 0.8567555004952836

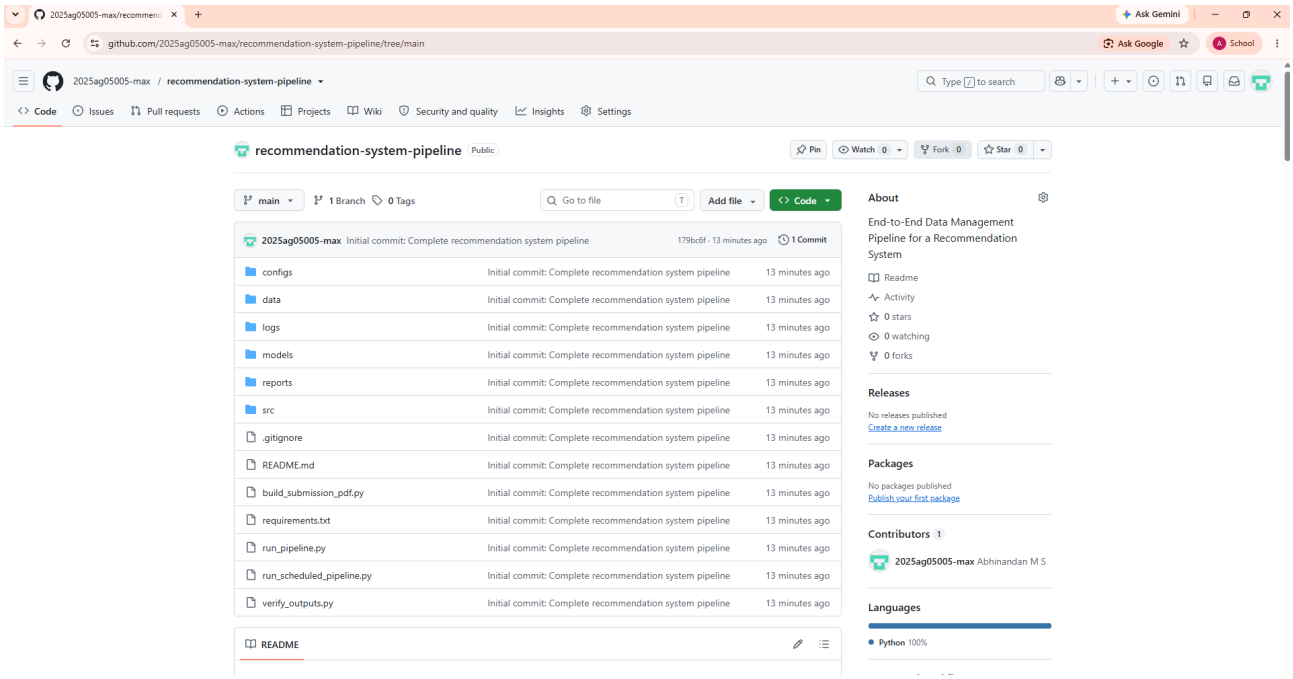
Model: ItemBasedCollaborativeFiltering
Run ID: 8f5c722f7e034800b166a06891c14555
precision_at_5 : 0.0
recall_at_5 : 0.0
ndcg_at_5 : 0.0
rmse : 1.033182613432211
mae : 0.804645625029475

Model: Popularity
Run ID: a19027ff82ce45ea9568faacd74d21e3
precision_at_5 : 0.009207161125319694
recall_at_5 : 0.036403494204005 71
ndcg_at_5 : 0.025593778300112557
rmse : 1.0743287145482574
mae : 0.8567555004952836

Model: ItemBasedCollaborativeFiltering
Run ID: d2ddf80dd8c6437cb000c66a47b35692
precision_at_5 : 0.0
recall_at_5 : 0.0
ndcg_at_5 : 0.0
rmse : 1.033182613432211
mae : 0.804645625029475

Model: Popularity
Run ID: 80137462de6743d3a5f4396dbe744697
precision_at_5 : 0.009207161125319694
recall_at_5 : 0.036403494204005 71
ndcg_at_5 : 0.025593778300112557
rmse : 1.0743287145482574
mae : 0.8567555004952836
(.venv) abhinandan@DESKTOP-O2SL2QK:~/recommendation-pipeline$
```

18. Public GitHub repository


2025ag05005-max/recommendation-system-pipeline
github.com/2025ag05005-max/recommendation-system-pipeline/tree/main
Ask Gemini
Ask Google
School
2025ag05005-max / recommendation-system-pipeline
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
recommendation-system-pipeline
Public
Pin
Watch 0
Fork 0
Star 0
main
1 Branch
0 Tags
Go to file
Add file
Code
2025ag05005-max Initial commit: Complete recommendation system pipeline
179bc6f · 13 minutes ago
1 Commit
configs Initial commit: Complete recommendation system pipeline 13 minutes ago
data Initial commit: Complete recommendation system pipeline 13 minutes ago
logs Initial commit: Complete recommendation system pipeline 13 minutes ago
models Initial commit: Complete recommendation system pipeline 13 minutes ago
reports Initial commit: Complete recommendation system pipeline 13 minutes ago
src Initial commit: Complete recommendation system pipeline 13 minutes ago
.gitignore Initial commit: Complete recommendation system pipeline 13 minutes ago
README.md Initial commit: Complete recommendation system pipeline 13 minutes ago
build_submission_pdf.py Initial commit: Complete recommendation system pipeline 13 minutes ago
requirements.txt Initial commit: Complete recommendation system pipeline 13 minutes ago
run_pipeline.py Initial commit: Complete recommendation system pipeline 13 minutes ago
run_scheduled_pipeline.py Initial commit: Complete recommendation system pipeline 13 minutes ago
verify_outputs.py Initial commit: Complete recommendation system pipeline 13 minutes ago
README
About
End-to-End Data Management Pipeline for a Recommendation System
Readme
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package
Contributors 1
2025ag05005-max Abhinandan M S
Languages
Python 100%